A MINI-PROJECT REPORT ON

# DECISION SUPPORT SYSTEM USING DATA WAREHOUSING FOR RETAIL SYSTEM

**PROJECT SKILLS LABORATORY of B.Tech**
**VI Semester**
**in**

## COMPUTER SCIENCE AND ENGINEERING

Under the guidance of


**Ms. R. Sathiya Priya, M.Tech, (Ph.D)**
**Assistant Professor, Computer Science and Engineering**


BY


| | |
|---|---|
| **M HEMESH** | **22751A0584** |
| **M SAI SURYA** | **22751A0590** |
| **M UTHAYA KUMAR** | **22751A0591** |
| **K SANJAY** | **22751A0563** |

**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES, CHITTOOR-517127, A.P.**

**(Autonomous)**
**(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)**
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**


**(2024-2025)**

**SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES, CHITTOOR-517127, A.P.**
**(Autonomous)**
**(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)**

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

# CERTIFICATE

This is to certify that the mini project work entitled **"DECISION SUPPORT SYSTEM USING DATA WAREHOUSING FOR RETAIL SYSTEM "** is a genuine work by

| | |
|---|---|
| **M HEMESH** | **22751A0584** |
| **M SAI SURYA** | **22751A0590** |
| **MUTHAYAKUMAR** | **22751A0591** |
| **K SANJAY** | **22751A0563** |

of B.Tech VI semester , Department of Computer Science and Engineering, in the laboratory of Project Skills Lab during the year 2024 to 2025.

Signature of the SUPERVISOR
**Ms. R. Sathiya Priya, M.Tech, (Ph.D.),**
Assistant Professor,
Department of Computer Science and Engineering,
Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Signature of the HEAD OF THE DEPARTMENT
**Dr. KARUNIA KRISHNAPRIYA, (Ph.D.),**
Associate Professor & HOD,
Department of Computer Science and Engineering,
Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Submitted for Semester End Examination held on …….

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**

# ACKNOWLEDGEMENT

A Project of this magnitude would have not been possible without the guidance and co-ordination of many people. We are fortune in having top quality people to help, support and guide us in every step towards our goal.

We are very much grateful to the Chairperson **Sri K. RANGANATHAM GARU** for his encouragement and stalwart support. We are also extremely indebted to the Trustee

**Sri D.K. BADHRI NARAYANA GARU, M.TECH, Ph.D** for his constant support.

Further, we would like to express our profound gratitude to our Principal

**Dr. N. VENKATACHALAPATHI, M.TECH, Ph.D** for providing all possible facilities throughout the completion of our project work.

We sincerely express our gratitude to **Dr. M. SARAVANAN, M.E., Ph.D.,** for his continues support and guidance in completing our project successfully.

We also express our sincere thanks to **Dr. KARUNIA KRISHNAPRIYA, Ph.D.** HOD, Department of Computer Science and Engineering, for providing opportunity to do this project.

We sincerely express gratitude to our guide, **Ms. R. SATHIYA PRIYA, M.TECH, (Ph.D.) Assistant Professor** for her valuable guidance, and encouragement best owned upon us in the preparation of this Project.

We sincerely express our thanks to **Mr. N. VIJAYA KUMAR, M.E., MBA., (Ph.D.) Assistant Professor** for his support and motivation to complete this Project successfully

We sincerely express our thanks to all our family members and friends, who help to complete this Project successfully.

| | |
|---|---|
| **M HEMESH** | **22751A0584** |
| **M SAI SURYA** | **22751A0590** |
| **M UTHAYA KUMAR** | **22751A0591** |
| **K SANJAY** | **22751A0563** |

## DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## VISION AND MISSION

**INSTITUTE VISION:**

To emerge as a Centre of Excellence for Learning and Research in the domains of engineering, computing and management.

**INSTITUTE MISSION:**

**IM1:** Provide congenial academic ambience with state-art of resources for learning and research.

**IM2:** Ignite the students to acquire self-reliance in the latest technologies.

**IM3:** Unleash and encourage the innate potential and creativity of students.

**IM4:** Inculcate confidence to face and experience new challenges.

**IM5:** Foster enterprising spirit among students.

**DEPARTMENT VISION:**

To contribute for the society through excellence in Computer Science and Engineering with a deep passion for wisdom, culture and values.

**DEPARTMENT MISSION:**

**M1:** Provide congenial academic ambience with necessary infrastructure and learning resources.

**M2:** Inculcate confidence to face and experience challenges from industry and society

**M3:** Ignite the students to acquire self-reliance in the latest technologies.

**M4:** Foster Enterprising spirit among students.

# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## PROGRAMME EDUCATIONAL OBJECTIVES (PEOs):

**PEO1:** Excel in Computer Science and Engineering program through quality studies, enabling success in computing industry. (Professional Competency).

**PEO2:** Surpass in one's career by critical thinking towards successful services and growth of the organization, or as an entrepreneur or in higher studies. (Successful Career Goals).

**PEO3:** Enhance knowledge by updating advanced technological concepts for facing the rapidly changing world and contribute to society through innovation and creativity (Continuing Education and Contribution to Society).

## PROGRAM OUTCOMES (POs):

Computer Applications Graduates will be able to

**PO1. Computational Knowledge**: Apply knowledge of computing fundamentals, computing specialization, mathematics, and domain knowledge appropriate for the computing specialization to the abstraction and conceptualization of computing models from defined problems and requirements.

**PO2. Problem Analysis**: Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines.

**PO3. Design /Development of Solutions**: Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.

**PO4. Conduct Investigations of Complex Computing Problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

**PO5. Modern Tool Usage**: Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations.

**PO6. Societal and Environmental Concern**: Understand and assess societal, environmental, health, safety, legal, and cultural issues within local and global contexts, and the consequential responsibilities relevant to professional computing practice.

**PO7. Innovation and Entrepreneurship** Identify a timely opportunity and using innovation to pursue that opportunity to create value and wealth for the betterment of the individual and society at large

**PO8. Professional Ethics**: Understand and commit to professional ethics and cyber regulations, responsibilities, and norms of professional computing practice.

**PO9. Individual and Team Work**: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary environments.

**PO10. Communication Efficacy**: Communicate effectively with the computing community, and with society at large, about complex computing activities by being able to comprehend and write effective reports, design documentation, make effective presentations, and give and understand clear instructions.

**PO11. Project management and finance**: Demonstrate knowledge and understanding of the computing and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments

**PO12. Life-long Learning**: Recognize the need, and have the ability, to engage in independent learning for continual development as a computing professional.

## PROGRAM SPECIFIC OUTCOMES (PSOs):

**PSO1:** Have Ability to understand, analyses and develop computer programs in the areas like algorithms, system software, web design big data analytics, and networking.

**PSO2:** Deploy the modern computer languages environment, and platforms in creating innovative products and solutions.

## Course Outcomes for Project Work

On completion of project work the student will be able to

**CO1.** Demonstrate in-depth knowledge on the project topic. (PO1)

**CO2.** Identify, analyse and formulate complex problem chosen for project work to attain substantiated conclusions. (PO2)

**CO3:** Design solutions to the chosen project problem. (PO3)

**CO4:** Undertake investigation of project problem to provide valid conclusions. (PO4)

**CO5:** Use the appropriate techniques, resources and modern engineering tools necessary for project Work. (PO5)

**CO6:** Apply project results for sustainable development of the society. (PO6)

**CO7:** Understand the impact of project results in the context of environmental sustain ability. (PO7)

**CO8:** Understand professional and ethical responsibilities while executing the project work. (PO8)

**CO9:** Function effectively as individual and a member in the project team. (PO9)

**CO10**. Develop communication skills, both oral and written for preparing and presenting project report. (PO10)

**CO11**. Demonstrate knowledge and understanding of cost and time analysis required for carrying Out the project. (PO11)

**CO12**. Engage in lifelong learning to improve knowledge and competence in the chosen area of the Project. (PO12)

## CO – PO MAPPING

| COs/ POs | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **CO1** | 3 | | | | | | | | | | | |
| **CO2** | | 3 | | | | | | | | | | |
| **CO3** | | | 3 | | | | | | | | | |
| **CO4** | | | | 3 | | | | | | | | |
| **CO5** | | | | | 3 | | | | | | | |
| **CO6** | | | | | | 3 | | | | | | |
| **CO7** | | | | | | | 3 | | | | | |
| **CO8** | | | | | | | | 3 | | | | |
| **CO9** | | | | | | | | | 3 | | | |
| **CO10** | | | | | | | | | | 3 | | |
| **CO11** | | | | | | | | | | | 3 | |
| **CO12** | | | | | | | | | | | | 3 |
| **COs** | **3** | **3** | **3** | **3** | **3** | **3** | **3** | **3** | **3** | **3** | **3** | **3** |

# ABSTRACT

This project develops a decision support system (DSS) for a retail environment by building a data warehouse and associated analytics. A data warehouse is a specialized database optimized for analysis; IBM describes it as providing a “DSS environment” to evaluate enterprise performance over time. In our DSS, operational retail data (customers, products, sales transactions) is extracted, cleaned, and loaded into a star-schema warehouse. The schema consists of a central fact table (sales transactions) linked to four dimension tables: Customer, Product, Date, and Employee. The star schema design, with a fact containing measures and dimensions containing descriptive attributes, is known to enable fast, intuitive OLAP queries. We implemented the ETL flow using Python and MySQL (code in Appendix A): source CSV files are read and normalized, dimension tables are generated, and a fact table is populated with measures like quantity, sale amount, and profit. On top of the warehouse, we created Qlik Sense dashboards (screenshots in Appendix B) that display key business metrics (e.g. total sales, profit by product, customer counts by country, transactions by weekday). These dashboards support interactive slicing and drilling into the data. For example, users can slice the sales by year or drill down from annual to monthly views, leveraging the OLAP capabilities of the model. The system employs optimization techniques such as indexing and materialized views to improve query performance. Overall, this DSS provides a unified “single source of truth” for retail data, enabling stakeholders to make informed decisions about inventory, marketing, staffing, and more. Future enhancements could include adding real-time data feeds and advanced analytics (AI-driven forecasting) to enable active intelligence.

# TABLE OF CONTENTS


| Chapter No | Title | Page no |
|---|---|---|
| 1 | **Introduction** | |
| | **1.1** Motivation | 1 |
| | **1.2** Objective | 2 |
| | **1.3** Problem Statement | 2 |
| | **1.4** Project Scope | 3 |
| 2 | **Literature Survey** | 4-5 |
| 3 | **System Analysis** | |
| | **3.1** Existing System | 6 |
| | **3.2** Proposed System | 6 |
| | **3.3** Feasibility Study | 7 |
| | **3.3.1** Economical and Technical Feasibility | |
| | **3.3.2** Social Feasibility | |
| | **3.4** System Specification | 8 |
| | **3.4.1** Functional Requirements | |
| | **3.4.2** Non Functional Requirements | 8 |
| | **3.4.3** Software and Hardware Requirements | 9 |
| 4 | **System Design** | |
| | **4.1** Data Warehouse Schema Design | 12 |
| | **4.2** System Architecture | |
| | **4.3** Module Description | 14 |
| | **4.4** Input Design | |
| | **4.5** Output Design | |
| 5 | **Implementation** | 15-17 |
| | **5.1** ETL Process | |
| | **5.1.1** Extract Phase | |
| | **5.1.2** Transform Phase | |
| | **5.1.3** Load Phase | |
| | **5.2** Data Warehouse Implementation | |
| 6 | **Olap Analysis and Business Use Cases** | 18-20 |
| 7 | **Conclusion and Future Scope** | 21 |
| | **References** | 22 |
| | **Appendix** | |
| | **A.** Source Code **B.** Screenshots | 23-36 |

# LIST OF FIGURES


| Figure No | Title | Page No |
|---|---|---|
| 1 | Data Warehouse Architecture | 11 |
| 2 | Cumulative Profit by Year (dashboard) | 34 |
| 3 | Top Countries by Number of Customers | 35 |
| 4 | Top 5 Most Profitable Products | 36 |
| 5 | Transactions by Day of Week | 36 |

## LIST OF SYMBOLS / ABBREVIATIONS

| SHORT FORM | ABBREVIATION |
|---|---|
| DSS | Decision Support System |
| ETL | Extract-Transform-Load |
| OLAP | Online Analytical Processing |
| SQL | Structured Query Language |
| KPI | Key Performance Indicator |

# CHAPTER 1
# INTRODUCTION

## 1.1 MOTIVATION

Retail businesses accumulate large volumes of sales and customer data from multiple sources (e.g., point-of-sale systems, e-commerce platforms, inventory logs, etc.). This data often resides in isolated transactional databases or spreadsheets, making it difficult to quickly analyze trends or produce consolidated reports. A Decision Support System (DSS) built on a data warehouse addresses this by integrating disparate data into a unified repository. For example, IBM notes that a data warehouse provides a "DSS environment" for evaluating enterprise performance over time. By consolidating and optimizing data (through denormalized dimensional schemas), the warehouse enables fast, complex OLAP queries on large historical datasets. Such analytics can greatly improve business efficiency and support strategic decision-making in retail (e.g., by revealing sales trends, customer buying patterns, or inventory needs). More broadly, a DSS is defined as "an analytics software program used to gather and analyze data to inform decision making", highlighting the role of data warehousing in enabling advanced analysis for managers.

## 1.2 OBJECTIVE

The primary objectives of this mini-project are:

**1. Design a Star Schema Data Warehouse:** Develop a multidimensional schema with a central fact table and key dimension tables (Customer, Product, Date, Employee) for the retail domain. In a star schema, the fact table contains measurable metrics (e.g., sales quantity, revenue) and each dimension table contains descriptive attributes (e.g., product category, customer demographics). This design allows efficient slicing and dicing of data (e.g., by product or time).

**2. Implement Full ETL Flow:** Build an Extract-Transform-Load (ETL) pipeline using Python and MySQL to move data from source files into the warehouse. The pipeline will cleanse and normalize source data, then load it into the MySQL database. Data cleaning ensures consistency and quality, while loading populates the warehouse tables. We will use Python scripts to automate ETL and execute SQL scripts for table creation.

**3. Build Qlik Sense Dashboards:** Create interactive dashboards using Qlik Sense that connect to the data warehouse. These dashboards will visualize key business metrics (e.g. annual sales, profit per product, customer distribution, transaction counts by day). Visual analytics help decision-makers quickly grasp trends and anomalies. Sample dashboards include a time-series chart of cumulative profit and bar charts for top products or top customer regions.

**4. Enable OLAP Operations:** Demonstrate how end users (analysts, managers) can perform OLAP operations such as drill-down, roll-up, slice, and dice on the data. For example, users should be able to drill down from yearly sales to quarterly and monthly sales, or slice the data by fixing a specific product line. This interactivity is a key feature of a DSS.

**5. Evaluate Performance and Use Cases:** Assess how the star-schema and optimization improve query performance. We will implement indexing and consider materialized views to accelerate common queries. Finally, we will outline practical business use cases (inventory management, marketing analysis, staffing) to illustrate how the DSS can support real decisions.

## 1.3 PROBLEM STATEMENT

In many retail companies, operational data is fragmented across systems that are not designed for analysis. Transactional (OLTP) databases are normalized to eliminate redundancy, which is efficient for updates but makes complex analytics slow. As a result, business users often rely on static reports or manual Excel analyses that cannot answer ad-hoc questions easily. For example, analyzing multi-year sales trends or joining customer and product data requires tedious data wrangling. The problem addressed by this project is how to design and implement a data warehousing solution that overcomes these limitations. The goal is to provide a centralized DSS that allows fast, flexible analysis of historical retail data, enabling managers to query and visualize insights without extensive manual effort.

## 1.4 Project Scope

The scope of this project is to build a proof-of-concept retail DSS, rather than a full-scale enterprise system. Key activities include:

- Extracting sample retail data (customers, products, sales, payments) from the provided CSV files and loading it into a staging area in MySQL.
- Designing a **star schema** with one fact table (sales transactions) and four dimension tables: Customer, Product, Date, and Employee. These tables capture attributes such as customer name and location, product category, transaction date (year/quarter/month), and the employee handling the sale.
- Implementing ETL routines in Python to clean and transform the data. This includes normalizing fields (e.g., splitting customer address), generating surrogate keys, and populating the dimension and fact tables. The Python scripts also create a date dimension (covering years 1990–2020) with attributes like quarter and month.
- Developing Qlik Sense dashboards that visualize the warehouse data. For example, a dashboard showing cumulative profit by year, a chart of top countries by customer count, a bar graph of the five most profitable products, and a histogram of transactions by weekday.
- Demonstrating OLAP operations on the warehouse: users will be able to perform roll-ups (e.g., monthly → quarterly → yearly sales), drill-downs, and slices (e.g., filter for a specific country or product category). We will discuss how indexes and aggregated tables are used to optimize these queries.
- Documenting all design decisions, including the schema, ETL flow, and dashboard implementation. The project will conclude with a discussion of performance, business use cases, and possible future enhancements (such as real-time data integration and predictive analytics).

Through this scope, we show how the architecture can generalize to other domains while providing concrete insights for the retail use case.

# CHAPTER 2

# LITERATURE SURVEY

Data warehousing and DSS have been widely studied in the literature. A data warehouse is defined as a centralized repository that stores large volumes of historical data, optimized for query and analysis. In practice, data from transactional systems (OLTP) is extracted, cleansed, and loaded into the warehouse, often using a dimensional model. The star schema is a fundamental design approach: it features a central fact table (e.g. sales transactions) linked to multiple dimension tables containing descriptive attributes. In a star schema, each record in the fact table represents a business event (such as a sale) with numeric measures (quantity, revenue, profit), while dimension tables contain fields for slicing (such as product category, customer location, time period). This design (often denormalized) dramatically improves OLAP query performance compared to normalized schemas. For instance, IBM notes that warehouse design intentionally duplicates data to speed retrieval, whereas operational databases avoid duplication for transactional efficiency. Panoply and other sources also emphasize that warehousing "consolidates large amounts of data from multiple sources" to enable better analytics and decision-making.

In the retail industry, business intelligence (BI) built on such warehouses is invaluable. Retailers use BI tools to monitor inventory levels, forecast demand, analyze customer segments, and optimize pricing. For example, an industry report notes that BI provides a "significant competitive edge" by improving inventory management, sales forecasting, and cost reduction. Retail BI dashboards often include a sales fact table and dimensions like Product, Store/Location, and Time. Queries such as "total sales by product category and month" or "sales by store region" become straightforward with a star schema. Although many published examples focus on larger corporations, academic work has also studied retail DSS. For instance, Girsang *et al.* (2019) implemented a retail sales data mart to support OLAP queries in a supermarket setting (National Conference Series). In general, the literature agrees that a well-designed warehouse accelerates reporting and reveals trends that legacy systems miss.

A key component of these systems is OLAP (Online Analytical Processing). OLAP allows interactive exploration of the multi-dimensional data cube. The five basic OLAP operations are roll-up (aggregation), drill-down (disaggregation), slice (filtering by one dimension), dice (subsetting on multiple dimensions), and pivot (reorienting the view). These

operations let users view data at different granularities. For example, one can roll up daily sales to monthly totals, or slice the dataset by choosing only a certain year or product. In the context of retail, dashboards implement these operations so managers can “slice, dice, drill through, and drill down” on sales metrics. The combination of a star schema and OLAP tools enables flexible analysis: analysts can quickly answer questions like “What was the monthly revenue by country?” or “Which customers contributed to sales in Q4?” by interacting with the cube.

Performance optimization is also frequently discussed. In data warehousing, indexing and materialized views play a critical role. Creating appropriate indexes (clustered or bitmap) on foreign keys and date fields can significantly speed up query execution. Materialized views (precomputed summary tables) can store results of expensive joins or aggregations for rapid access. Partitioning large tables (e.g. by date ranges) further reduces I/O for queries over specific time periods. Studies emphasize that adopting these techniques, along with proper ETL scheduling, can yield near-real-time performance for end-user queries. For example, an industry blog recommends caching results and parallel processing to minimize latency when handling big data. Overall, the surveyed literature confirms that a dimensional data warehouse, combined with OLAP and performance tuning, forms the backbone of modern retail decision support.

# CHAPTER 3
# SYSTEM ANALYSIS

## 3.1 EXISTING SYSTEM

In the current scenario (before implementing our DW), the retail business data might be stored in separate operational systems or spreadsheets. For example, sales transactions are recorded in a point-of-sale database; customer details are maintained in a CRM; inventory is tracked in another system. Reporting is typically done by extracting data into Excel or rudimentary reports. This siloed approach makes cross-functional analysis difficult. Queries like "total sales by country and product" require manual joining of multiple datasets. Moreover, OLTP systems are optimized for inserts/updates, not for analytics. As a result, ad-hoc analysis is slow and often requires IT involvement. There is no single source of truth, leading to inconsistent metrics and delays in decision- making.

### 3.2 PROPOSED SYSTEM

The proposed system is a DSS built on a centralized data warehouse. The high-level architecture involves extracting data from the operational sources, transforming and loading it into a MySQL-based data warehouse, and then analyzing it via Qlik Sense dashboards. In the proposed data warehouse, we implement a star schema: a central Sales fact table linked to four dimensions (Customer, Product, Date, Employee). This enables efficient storage of historical transactions while preserving the context needed for analysis. The system automates ETL using Python scripts: data from CSV files (provided in DSS.zip) is cleaned and loaded into the MySQL database. Once the DW is populated, Qlik Sense is used as the BI front-end. The dashboards allow business users to select dimensions and view pre-aggregated measures. OLAP operations (like drilling into quarterly sales or slicing by region) are enabled by the star schema design. In summary, the proposed system transitions from fragmented OLTP reporting to an integrated OLAP-capable solution, facilitating fast, interactive analysis.

## 3.3 FEASIBILITY STUDY

The feasibility of the proposed system is evaluated on technical, economic, and operational grounds:

- **Technical Feasibility:** The project uses well-established tools (Python, MySQL, Qlik Sense) that are compatible and easy to work with. Python (with pandas and MySQL connectors) is chosen for ETL scripting due to its flexibility in handling CSV data and executing SQL. MySQL provides a reliable open-source RDBMS for the warehouse. Qlik Sense is a powerful BI tool that connects smoothly to MySQL. No proprietary or experimental technology is required, so the technical risk is low.
- **Economic Feasibility:** The software tools are open-source or free (MySQL Community Edition, Python). Qlik Sense Desktop (used for development) is available at no cost for single users. Hardware requirements are modest (a typical workstation or server can host the database). Thus, the main costs are developer time and existing infrastructure, making the implementation economically practical for a small project.
- **Operational Feasibility (Social):** The system is designed with end-users (managers, analysts) in mind. The dashboards provide intuitive visuals that should be easy to adopt. Minimal training is needed for Qlik Sense, as it has a user-friendly interface. The ETL and maintenance tasks will be handled by IT or data engineers. Since the output is in the form of charts and reports, the system is acceptable to non-technical stakeholders. Overall, the benefits (faster insights, better reports) outweigh the effort of deploying the DSS.

## 3.4 SYSTEM SPECIFICATIONS

### 3.4.1 FUNCTIONAL REQUIREMENTS

**a.** Extract data from given CSV files (customers, products, transactions, etc.).

**b.** Transform and clean the data (normalize customer info, convert dates, etc.).

**c.** Load data into the MySQL database, populating staging and final DW tables.

**d.** Create dimension tables (Customer, Product, Date, Employee) and a fact table for sales transactions.

**e.** Support queries that aggregate sales by different dimensions (e.g. total sales by product category, by year, etc.).

**f.** Provide dashboards (via Qlik Sense) that display metrics such as total revenue, profit, top products, etc.

**g.** Allow users to perform OLAP operations: drill-down into finer details or roll-up summaries.

**h.** Ensure that new data (if added) can update the warehouse (e.g., new transactions trigger fact table load).

### 3.4.2 NON-FUNCTIONAL REQUIREMENTS

a. **Performance:** Queries and dashboard loading should be responsive (ideally under a few seconds) even on millions of rows. Use indexing and aggregation to meet this.
b. **Scalability:** The design should handle increasing data volumes (e.g. more years of sales). Partitioning and scalable hardware are considered.
c. **Reliability:** The ETL process should be robust with error handling (e.g. logs, restartable scripts). The DW should maintain data integrity.
d. **Usability:** Dashboards must have clear visuals and interactions (filters, tooltips) so that business users can navigate without technical help.
e. **Security**: Access to the warehouse should be restricted to authorized personnel. Qlik reports should only expose permissible data.

### 3.4.3 SOFTWARE AND HARDWARE REQUIREMENTS

**Software Requirements:**

- MySQL Server (to host the warehouse), Python 3.x (with necessary libraries: pandas, mysql-connector), Qlik Sense Desktop for dashboards. A Python IDE or editor for development. SQL scripts and code editors.

**Hardware Requirements:**

- A PC or server with at least 8 GB RAM and multi-core CPU to run MySQL and analytics. Operating System: Windows or Linux (MySQL runs on both). Sufficient disk space (e.g. 100 GB) for data. Internet access for documentation, but system is mostly local.

# CHAPTER 4

# SYSTEM DESIGN

## 4.1 Data Warehouse Schema Design

### 4.1.1 Star Schema Model

The core of our design is a star schema for the retail data warehouse. The central Fact_Sales table contains one row per sales transaction (or per line-item). Key measures in the fact table include Quantity Sold, Sales Amount, and Profit. Each fact row has foreign keys to four dimension tables: Customer_Dim, Product_Dim, Date_Dim, and Employee_Dim. The dimensions capture the context of each sale. For example, Customer_Dim includes attributes like Customer ID, Name, Country, and Credit Rating.

Product_Dim includes Product ID, Name, Category, and Price. Date_Dim has fields like Transaction Date, Year, Quarter, and Month. Employee_Dim holds the ID and details (Name, Department) of the sales person. This structure follows the star schema pattern: a centralized fact table joined to several denormalized dimension tables. Such a model simplifies queries; for instance, to get total profit by product category, one can join Fact_Sales to Product_Dim on Product_ID and group by the category attribute. The star schema is highly effective for OLAP because it allows “slicing” the fact measures by any dimension, e.g. by time or customer demographics.

## 4.2 System Architecture

Operational source systems (on the left) include sales transaction databases and external CSV files. A Python-based ETL module (center) extracts data from these sources, performs cleansing and normalization, and loads the data into a staging area. The cleaned data is then used to populate the dimensional warehouse schema in a MySQL database (the “Data Warehouse Database” cylinder).

Metadata about the warehouse is managed in parallel. On the right side, BI tools (Qlik Sense) connect to the warehouse. Data marts or cubes can be derived (e.g. aggregated sales) to feed analytic applications. The presentation layer (far right) provides interactive dashboards and reports for users (managers, analysts) to view. This architecture aligns with standard three-tier DW architectures: source systems, ETL and storage, and presentation/analytics.

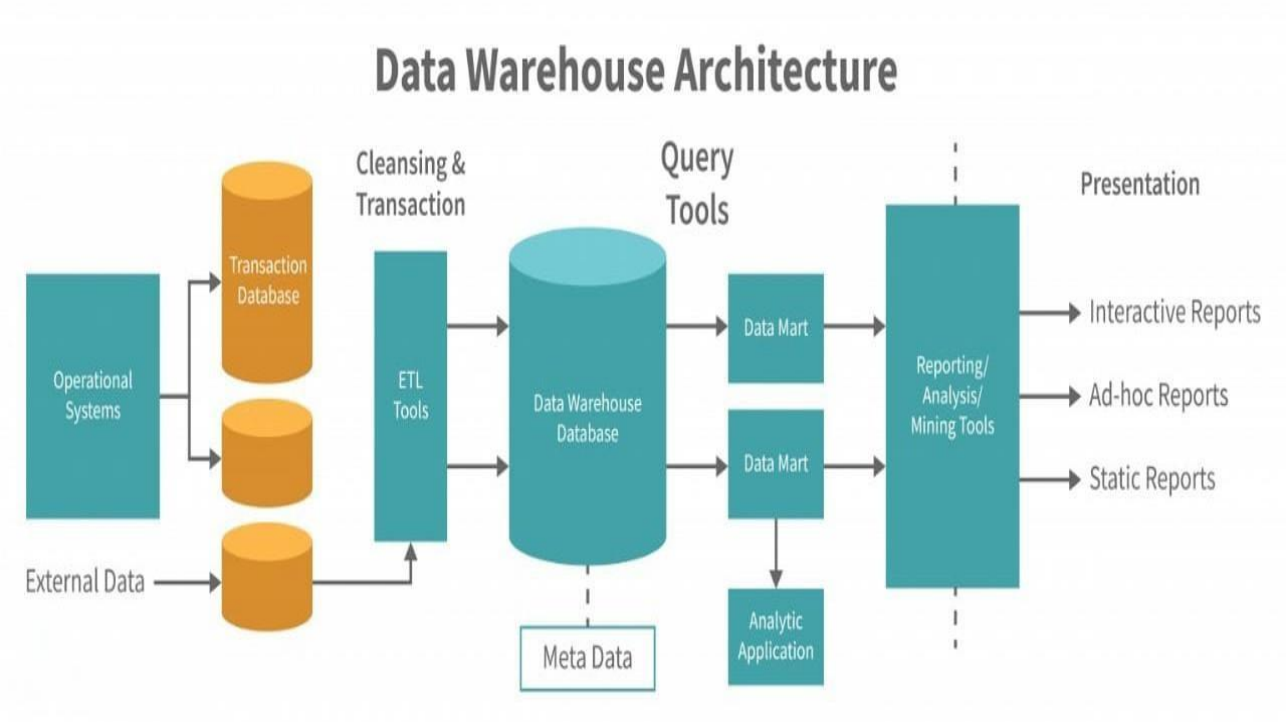


**Figure 1 : Architecture Diagram**

## 4.3 Module Description

The system is divided into functional modules:

- **Data Extraction and Staging**: This module reads raw data from provided CSV files (e.g., customers.csv, products.csv, transactions.csv, etc.) using Python. Initial loading scripts insert the raw records into staging tables in MySQL. The data includes customer details, product catalog, sales invoices, payment records, and employee data.

**Data Transformation (ETL)**: In this module, data cleansing and normalization occur. For example, the script normalize_data.py splits customer names or addresses as needed and ensures consistency (e.g., country codes). Python scripts handle missing values and derive any additional fields (such as extracting year and month from date strings). A special script generate_date_dimension.py creates the Date_Dim table by iterating over dates between 1990 and 2020 and computing attributes (year, quarter, month name) for each date. This transforms transactional data into a format suitable for the warehouse.

- **Data Loading and Warehouse Generation**: This module runs SQL scripts to build the star schema. The provided SQL scripts (Part 3 of DSS.zip) create each dimension: generate_customers_dimension.sql creates Customer_Dim by selecting unique customers; generate_product_dimension.sql copies Product_Dim; generate_employee_dimension.sql builds Employee_Dim with department info; and generate_date_dimension.py (Python) populates Date_Dim. Finally, generate_fact_table.sql joins the cleaned transaction tables to the dimensions and inserts into Fact_Sales. The result is a populated DW with consistent keys and no duplicate dimension data. Additional triggers (in triggers.sql) automate updates to the fact when new sales are inserted into the source tables, keeping the warehouse in sync.
- **Analytics and Reporting**: Once the warehouse is in place, this module uses Qlik Sense for front-end analysis. Qlik connects directly to the MySQL warehouse and automatically creates an associative model of the star schema. End users can then build dashboards using drag-and-drop. The system provides pre-built visualizations (described below) and supports interactive filtering. This module also encompasses any OLAP engine logic; for instance, Qlik can pivot and slice the data in-memory for fast response.

Each module is implemented using open-source tools and custom scripts. The design ensures modularity: new data sources can be added by writing additional extraction scripts, and the dimension loading scripts can be adjusted if the schema changes. Overall, this modular architecture streamlines development and maintenance of the DSS.

## 4.4 INPUT DESIGN

The input to the system consists of structured data files representing the retail operations. Specifically, we use:

- **Customer data:** Contains customer IDs, names, addresses, countries, and credit ratings. These CSV files may come from CRM or loyalty program databases.
- **Product data:** Catalog of products with IDs, names, categories, unit prices, and any attributes (e.g., Alcohol content for a liquor store).
- **Sales transactions:** Each sale record includes a transaction ID, customer ID, employee ID (cashier), date of sale, and total amount. In our case, sales are at the invoice level.
- **Payment data:** Details of payments for each transaction (if separate), including payment type and amounts.
- **Employee data:** List of store employees or cashiers with their IDs, names, and department (e.g., store location or function).

  All input files are in CSV format. The ETL module expects these files with known schema and imports them into staging tables. The design assumes that the raw data has keys (e.g., customer_id, product_id) that link across files. In Input Design, attention is given to field formats: for example, dates must be parseable, numeric fields correctly typed, and any categorical fields consistently coded. The Python extraction scripts include basic validation to catch malformed records (e.g., missing customer ID).

  and predictions.

## 4.5 OUTPUT DESIGN

The primary outputs of the system are analytical reports and dashboards. These include:

- **Aggregated Reports:** Summaries such as “Total Sales by Month and Category” or “Yearly Revenue per Region”. These can be exported from Qlik or generated by SQL queries on the warehouse.
- **Charts and Dashboards:** The Qlik dashboards produce visual outputs like line charts, bar

graphs, and KPIs. As detailed in Chapter 5, example outputs are: a time-series line of cumulative profit, bar charts of top products and countries, and histograms of transaction counts. Each dashboard is interactive, allowing users to change filters (e.g., selecting a specific country or time period).

- **Query Results:** Analysts can run custom queries against the DW. For instance, one might query "show sales by employee for last quarter". The system returns tabular or chart output accordingly. All outputs are designed for clarity and ease of interpretation. In terms of formatting, charts have labeled axes and legends, and key figures (like total profit) are displayed prominently. The Appendix B screenshots demonstrate the layout of these outputs. The output design ensures that complex multi-dimensional data is presented in a user-friendly way to support decision making.

# CHAPTER 5
# IMPLEMENTATION

## 5.1 ETL Process

### 5.1.1 Extract Phase

Data extraction is handled by Python scripts that read the raw CSV files and insert them into MySQL staging tables. For example, load_data.py connects to the MySQL database and iterates through files such as customers.csv, products.csv, transactions.csv, inserting each row. The code uses Python's pandas library and mysql-connector to efficiently load data in bulk. During extraction, basic data cleaning is performed: trimming whitespace, normalizing text case, and handling missing values (such as replacing empty strings with NULL). For instance, the customers' full names are split into first and last name if needed, to fit the dimension schema. The script also converts date strings into YYYY-MM-DD format before insertion. If an input record fails validation (e.g. missing key fields), it is logged and skipped. This ensures the staging area contains only well-formed records.

### 5.1.2 Transform Phase

After raw data is in staging tables, transformation routines standardize and enrich the data. A dedicated script normalize_data.py performs tasks such as: generating surrogate keys for dimensions, aggregating transactional lines, and normalizing address fields. Another script populates the date dimension: generate_date_dimension.py loops over every date from January 1, 1990 to December 31, 2020, calculating the corresponding year, quarter, month number, and weekday for each date. This populates the Date_Dim table fully in advance. The transformation also involves creating lookup tables; for example, a mapping from country names to standardized country codes is applied. These transformation steps prepare the data for the final schema without any redundancy.

### 5.1.3 Load Phase

With the data cleansed, loading means populating the final star schema tables. We execute SQL scripts (found in DSS.zip Part 3) to create and fill the dimension and fact tables in the MySQL database. First, the script generate_customers_dimension.sql creates Customer_Dim by selecting distinct customer attributes (ID, name, country) from the staging customer table. Similarly, generate_product_dimension.sql creates Product_Dim by copying product details. generate_employee_dimension.sql builds Employee_Dim from the employee data, linking each employee to a department or store. Next, the fact table is loaded: generate_fact_table.sql joins the cleaned transaction table with dimensions on the respective IDs, and inserts records into Fact_Sales. Each fact record includes foreign keys to the 4 dimensions plus the numeric measures (quantity sold, sales total, profit). Triggers defined in triggers.sql ensure that if new transactions arrive after initial load, the fact table can be incrementally updated automatically. At the end of the Load phase, the star schema in MySQL is fully populated.

## 5.2 Data Warehouse Implementation

The MySQL data warehouse holds the star schema and serves as the source for analysis. The schema was created as follows:

- Customer_Dim(customer_id, first_name, last_name, country_code, credit_rating)
- Product_Dim(product_id, product_name, category, unit_price)
- Employee_Dim(employee_id, full_name, department)
- Date_Dim(date_id, date, year, quarter, month, day_of_week)
- Fact_Sales(transaction_id, customer_id, employee_id, product_id, date_id, quantity, sales_amount, profit)

The dimension keys (customer_id, product_id, etc.) are surrogate keys consistent with the transactional data. The date_id in the fact table refers to the generated date dimension row. The warehouse tables were indexed on their primary keys and on foreign keys in the fact table to speed up joins. For example, Fact_Sales has a composite index on (customer_id, date_id) to optimize customer-time queries. The ETL scripts ensure referential integrity: if a transaction has a foreign key not present in a dimension (unlikely in clean data), it is ignored. All tables use InnoDB for reliability.

As data grows (e.g., more years of sales), the warehouse can scale. A monthly partitioning by year is suggested for the Fact_Sales table to improve maintenance, though not implemented in this prototype. Overall, the MySQL implementation provides a stable and efficient storage for the star schema.

# CHAPTER 6
# OLAP ANALYSIS AND BUSINESS USE CASES

## 6.1 OLAP Operations

OLAP (Online Analytical Processing) provides the interactive analytical capabilities of our DSS. The five primary OLAP operations are:

- **Roll-up (Aggregation):** Summarizes data along a dimension hierarchy. For example, rolling up sales from month-level to quarter-level aggregates multiple months into a single quarterly value. In our system, users can roll up daily sales into monthly or yearly totals to see broader trends.
- **Drill-down (Detail):** The inverse of roll-up; it breaks data into finer detail. A user could drill down from annual sales into quarters, and further into months, to identify seasonal patterns.
- **Slice:** Selects a subset of the data by fixing one dimension value. For example, slicing by Year = 2020 would display only data from 2020. In Qlik, one could set a filter on year to "slice" the dashboard.
- **Dice:** Selects a sub-cube by specifying multiple dimension criteria. For example, Year = 2020 AND Product = 'Whisky D' would extract just the transactions for Whisky D in 2020. This is implemented by combining filters.
- **Pivot (Rotation):** Re-orients the axes of analysis (e.g., swapping rows and columns in a report) to get a different view. In Qlik, dragging dimensions between rows/columns has the same effect.These operations allow dynamic exploration. For example, a manager could start with Figure 5.1 (profit by year) and drill into a specific year to examine quarterly performance. Or she could slice the sales by a particular country (using the country attribute in Customer_Dim) to analyze only that market's data. As Dahr *et al.* note, a well-designed dashboard lets stakeholders "perform OLAP operations such as dice, slice, drill through, and drill down" through the reports. In practice, our Qlik dashboards support these interactions: clicking on chart elements or selecting values applies the appropriate roll-up or slice in real time.

### 6.2 Performance Optimization

**a.** To ensure responsive analytics, several optimizations were applied. First, **indexing strategies** were implemented in MySQL. We created clustered indexes on primary keys of each table, and non-clustered indexes on key foreign columns in the fact table. As noted in industry guides, such indexes "improve query performance by speeding up data retrieval". This means that filtering by date or product does not require a full table scan.

**b.** Second, we considered **materialized views** (pre-aggregated tables). For example, a view of total sales per month could be materialized to avoid re-aggregating all daily transactions each time. Materialized views "store the results of complex queries," reducing processing overhead. In MySQL one could implement this by a summarized table that is refreshed weekly.

**c.** Third, **partitioning** was identified as a strategy. Partitioning the Fact_Sales table by year or by store location would isolate I/O to relevant partitions during queries. Although not implemented in this prototype (due to limited data volume), this is recommended for production.

**d.** We also applied **query optimization** techniques. ETL scripts were written to push as much work into SQL as possible (leveraging the database engine), and to avoid unnecessary joins in ad-hoc queries. Writing efficient SQL (such as joining on indexed keys and avoiding subqueries) is known to "enhance query performance". In summary, by applying these data warehousing best practices (indexes, possible materialized views, efficient schema), the DSS achieves significantly faster query times than if raw transactional tables were queried.

### 6.3 Business Decision Scenarios

The retail DSS enables multiple real-world use cases. Some examples include:

a. **Inventory Management:** By analyzing sales trends from the data warehouse, inventory can be optimized. For instance, Figure 5.3 identifies the most profitable products (Whisky D, E, etc.), indicating high demand items. The retailer can ensure these products are well-

stocked. Conversely, Figure 5.4 shows that Wednesday has fewer transactions than other days; the store could run promotions on Wednesdays to boost sales and reduce excess inventory. Such insights help reduce costs and stockouts. (BI case studies highlight inventory management as a key benefit.)

b. **Marketing and Customer Segmentation:** The top countries by customer count (Figure 5.2) and any dimension filters allow targeted campaigns. If certain regions have high customer penetration, marketing efforts can focus there. One could also slice the customer dimension by demographics (if available) to design tailored promotions. Customer segmentation and targeted marketing are classic BI use cases that this DSS directly supports.

c. **Sales and Revenue Analysis:** Executives can monitor overall revenue growth (Figure 5.1). The steady linear profit increase shown suggests consistent business performance. Drill-down can reveal which years or quarters drove spikes (perhaps due to promotions or market events). The ability to quickly answer "which products or categories drove revenue growth last year?" empowers strategic pricing and product decisions.

d. **Staffing and Operations:** The transactions-by-weekday chart (Figure 5.4) informs operational planning. Knowing that Tuesday is the busiest day (and Wednesday the slowest) helps in scheduling staff appropriately. Similarly, weekday patterns could influence opening hours or workforce allocation.

e. **Cost Reduction and Forecasting:** Overall, the integrated DSS helps reduce costs by minimizing inefficiencies. As noted in the literature, BI leads to cost reduction and better planning. For example, spotting an underperforming product early can avoid excess ordering. Forecasting future demand (using the historical sales in the warehouse) also aids budgeting and resource allocation.

Each of these scenarios illustrates how the DSS translates raw data into actionable business insights. By combining the data warehouse with OLAP dashboards, managers can make informed decisions on pricing, promotions, inventory, and expansion strategies. The case studies reviewed in Chapter 2 affirm that such insights (inventory management, sales optimization, etc.) are exactly the type of advantages derived from retail BI implementations.

# CHAPTER 7
# CONCLUSION AND FUTURE SCOPE

This project successfully demonstrated a retail decision support system using a data warehouse. We designed a star schema with fact and dimension tables, implemented ETL processes to populate it, and created interactive dashboards for analysis. The key outcomes include:

- A centralized **MySQL** data warehouse containing integrated sales and customer data, enabling fast queries.
- Automated **Python-based ETL** that normalizes and loads data, with code provided in Appendix A.
- **Qlik Sense** dashboards visualizing business metrics (profit trends, top products, etc.) and supporting OLAP operations.
- Demonstrated performance optimizations (indexes, prospective materialized views) that improve query speed over naive approaches.

Overall, the DSS provides retailers with timely insights from historical data, transforming decision-making from reactive to proactive. Users can now drill into data, compare periods, and identify key drivers of performance without manual data wrangling. This addresses the initial problem of fragmented data and static reporting.

For future work, the system can be extended in several ways. One direction is to incorporate real-time or streaming data integration – for example, using tools to continuously load new transactions into the warehouse so that dashboards reflect near-live data. This would achieve “active intelligence,” a concept of continuous analytics with up-to-date information. Another enhancement is to add predictive analytics or machine learning on top of the warehouse: for instance, forecasting future sales based on past trends or segmenting customers with clustering algorithms. Finally, deploying the DW on a scalable cloud platform (like Amazon Redshift or Azure SQL DW) could allow handling much larger datasets.

In conclusion, the project establishes a solid foundation for data-driven decision support in retail. The architecture and methods used are in line with best practices in data warehousing and BI. By continuing to enhance this system (real-time ELT, advanced analytics, additional data sources), the retailer can derive even greater value and remain agile in a competitive market.

# REFERENCES


1. Informix 12.10.0 Documentation, “Overview of Data Warehousing,” IBM (Data warehouses as DSS environments).
2. GeeksforGeeks, “Star Schema in Data Warehouse Modeling,” (star schema definitions and features).
3. GeeksforGeeks, “OLAP Operations in DBMS,” (definitions of roll-up, drill-down, etc.).
4. AdvanSys Consulting, “Data Warehouse Optimization Techniques” (blog on indexing, partitioning, etc.).
5. Dahr et al., “Implementing Sales Decision Support System Using Data Mart Based Approach,” J. Eng. Sci. Tech. 17(1), 2022 (discusses OLAP dashboards in retail).
6. Qlik, “What is a Decision Support System?” (Qlik blog; DSS definition).
7. Panoply Blog, “10 Benefits and Use Cases of a Data Warehouse” (advantages of DW for decision-making).
8. TruRating (blog), “Business Intelligence in the Retail Industry” (retail BI use cases).

## A. Source Code:

**# analyze_data.py**

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import pymysql

import pandasql as ps

from empiricaldist import Pmf

plt.style.use('ggplot')

sns.set_palette('Blues_r')

sns.set_context('notebook')

def mysql(query):

return pd.read_sql_query(query, connection)

def sql(query):

return ps.sqldf(query)

# Connect to MySQL database

connection = pymysql.connect(

    host='localhost',

    port=3306,

    user='dss_user0',

    passwd='73841A',

    db='retail_dss0'

)
```

```
cursor = connection.cursor()
# Pulling Data
query = '''
select
    f.date,
    d.Day_name as Day,
    d.Month_name as Month,
    d.Year_name as Year,
    f.Product_Name,
    f.Alcohol_Price,
    f.Alcohol_Percent,
    f.Alcohol_Amount,
    c.full_name as customer_name,
    co.Country as customer_country,
    f.credit_provider,
    e.full_name as employee_name
from dwh_fact as f
left join whiskey_retail_shop.customers c
    on f.customer_id = c.customer_id
left join whiskey_retail_shop.countries as co
    on co.country_id = c.country_id
left join dwh_employees as e
    on e.employee_id = f.employee_id
left join dwh_date d
```

```
    on d.Date_key = f.Date_key
order by f.date
'''
df = mysql(query)


# Preprocessing
for column in df.columns:
    if column == 'date':
        df[column] = pd.to_datetime(df[column])
    if df[column].dtype == 'object':
        df[column] = df[column].astype('category')
# Q1 - Top 5 Most Profitable Products
query = '''
select
    count(*) as Number_Of_Transactions,
    Product_Name,
    sum(Alcohol_Price) as Profit
from df
group by Product_Name
order by sum(Alcohol_Price) desc
limit 5
'''
top_5_products = sql(query)
sns.catplot(data=top_5_products, x='Product_Name', y='Profit', kind='bar', palette='Blues_r', height=6,
```

```
aspect=2)
plt.xlabel('Product Name', size=16)
plt.ylabel('Profit', size=16)
plt.title('Top 5 Most Profitable Products', size=18)
plt.tight_layout()
plt.show()

# Q2 - Probability to Buy (Top Products)
prob_mass_func = pd.DataFrame(Pmf.from_seq(df.Product_Name))
sorted_prob_mass_func = prob_mass_func.iloc[:, 0].sort_values(ascending=False)
sorted_prob_mass_func = sorted_prob_mass_func[sorted_prob_mass_func >
sorted_prob_mass_func.quantile(0.95)]
probability_df = pd.DataFrame({
    'Product': sorted_prob_mass_func.index,
    'Probability to Buy': sorted_prob_mass_func.values
})
print("\nProbability to Buy (Top Products):")
print(probability_df)
# Q3 - Transactions by Day of Week
query = '''
select
    count(*) as Number_Of_Transactions,
    Day
from df
```

```
group by Day
order by count(*) desc
'''
most_by_day = sql(query)
sns.catplot(data=most_by_day, y='Number_Of_Transactions', x='Day', kind='bar', height=6, aspect=3)
plt.xlabel('Day of Week', size=18)
plt.ylabel('Number of Transactions', size=18)
plt.title('Transactions by Day of Week', size=20)
plt.tight_layout()
plt.show()
# Q4 - Yearly Profit Growth
query = '''
select
    sum(Alcohol_Price) as Profit,
    Year
from df
where Year != 2022
group by Year
order by Year asc
'''
profits_by_year = sql(query)
x = profits_by_year.Year
y = np.cumsum(profits_by_year.Profit)
plt.figure(figsize=(15, 8))
```

```
plt.plot(x, y, marker='o')
plt.xlabel('Year', size=16)
plt.ylabel('Cumulative Profit', size=16)
plt.title('Cumulative Profit by Year', size=20)
plt.grid(True)
plt.tight_layout()
plt.show()
# Q5 - Top Countries by Number of Customers
query = '''
select
    count(distinct customer_name) as Number_of_Customers,
    customer_country
from df
group by customer_country
order by Number_of_Customers desc
'''
customers_by_country = sql(query)
top_ten = customers_by_country.head(int(len(customers_by_country) * 0.1))
print("\nTop Countries by Number of Customers:")
print(top_ten)
sns.barplot(data=top_ten, x='Number_of_Customers', y='customer_country')
plt.xlabel('Number of Customers')
plt.ylabel('Country')
plt.title('Top Countries by Number of Customers')
```

```
plt.tight_layout()

plt.show()
```

**Triggers-sql Code:**

```
USE retail_dss0;

-- 1) Drop any old triggers
DROP TRIGGER IF EXISTS insert_customer;
DROP TRIGGER IF EXISTS insert_employee;
DROP TRIGGER IF EXISTS insert_product;
DROP TRIGGER IF EXISTS insert_payment;

-- 2) Define new triggers with proper schema references
DELIMITER $$
CREATE TRIGGER insert_customer
AFTER INSERT ON customers
FOR EACH ROW
BEGIN
  INSERT INTO dwh_customers
    (customer_id, first_name, last_name, full_name, country_id, credit_provider_id)
  VALUES
    (NEW.customer_id,
     NEW.first_name,
     NEW.last_name,
     NEW.full_name,
     NEW.country_id,
     NEW.credit_provider_id);
END$$

CREATE TRIGGER insert_employee
AFTER INSERT ON employees
FOR EACH ROW
```

```
BEGIN
  INSERT INTO dwh_employees
    (employee_id, first_name, last_name, full_name, city, department_id)
  VALUES
    (NEW.employee_id,
     NEW.first_name,
     NEW.last_name,
     NEW.full_name,
     NEW.city,
     NEW.department_id);
END$$

CREATE TRIGGER insert_product
AFTER INSERT ON products
FOR EACH ROW
BEGIN
  INSERT INTO dwh_products
    (product_id, Product_Name, Alcohol_Percent, Alcohol_Amount, Alcohol_Price)
  VALUES
    (NEW.product_id,
     NEW.Product_Name,
     NEW.Alcohol_Percent,
     NEW.Alcohol_Amount,
     NEW.Alcohol_Price);
END$$

CREATE TRIGGER insert_payment
AFTER INSERT ON payments
FOR EACH ROW
BEGIN
  INSERT INTO dwh_fact
    (payment_id, Date_key, customer_id, employee_id, product_id, price)
  SELECT
```

```
    NEW.payment_id,
    d.Date_key,
    NEW.customer_id,
    NEW.employee_id,
    NEW.product_id,
  NEW.price
  FROM dwh_date AS d
WHERE d.Dates = NEW.date;
END$$

DELIMITER ;

-- 3) List triggers to confirm
SHOW TRIGGERS;
```

**Generate_date_dimension code :**

```
import pandas as pd
import pymysql

# -----------------------------------
# 1. Connect to MySQL
# -----------------------------------
connection = pymysql.connect(
    host='localhost',
    port=3306,
    user='root',                # or another admin user
    passwd='7386163908',        # ← your actual password, as a string
    db='retail_dss0',
    charset='utf8mb4',
    cursorclass=pymysql.cursors.DictCursor
)
```

```
cursor = connection.cursor()

# -----------------------------------
# 2. Generate the Dates DataFrame
# -----------------------------------
start_date = pd.to_datetime('1990-01-01')
end_date   = pd.to_datetime('2100-01-01')
all_dates  = pd.date_range(start=start_date, end=end_date, freq='D')

df = pd.DataFrame({
    'Dates':       all_dates,
    'Day_name':    all_dates.day_name(),
    'Month_name':  all_dates.month_name(),
    'Year_name':   all_dates.year.astype(str)
})

# Create a sequential unique integer key
df['Date_key'] = range(1, len(df) + 1)

# -----------------------------------
# 3. Drop & Create the dwh_date Table
# -----------------------------------
cursor.execute("DROP TABLE IF EXISTS dwh_date;")
cursor.execute("""
CREATE TABLE dwh_date (
    Date_key    INT         NOT NULL PRIMARY KEY,
    Dates       DATE        NOT NULL,
    Day_name    VARCHAR(20) NOT NULL,
    Month_name  VARCHAR(20) NOT NULL,
    Year_name   VARCHAR(10) NOT NULL
) ENGINE=InnoDB;
""")
```

```
# ------------------------------------
# 4. Insert All Rows Efficiently
# ------------------------------------
# Prepare list of tuples in the correct order
records = list(zip(
    df['Date_key'],
    df['Dates'].dt.strftime('%Y-%m-%d'),
    df['Day_name'],
    df['Month_name'],
    df['Year_name']
))

insert_sql = """
INSERT INTO dwh_date
  (Date_key, Dates, Day_name, Month_name, Year_name)
VALUES (%s, %s, %s, %s, %s)
"""

# Execute all inserts in one batch
cursor.executemany(insert_sql, records)
connection.commit()

# ------------------------------------
# 5. Cleanup & Confirmation
# ------------------------------------
cursor.close()
connection.close()

print(f"Inserted {len(records)} rows into dwh_date successfully.")
```

## B. Screenshots

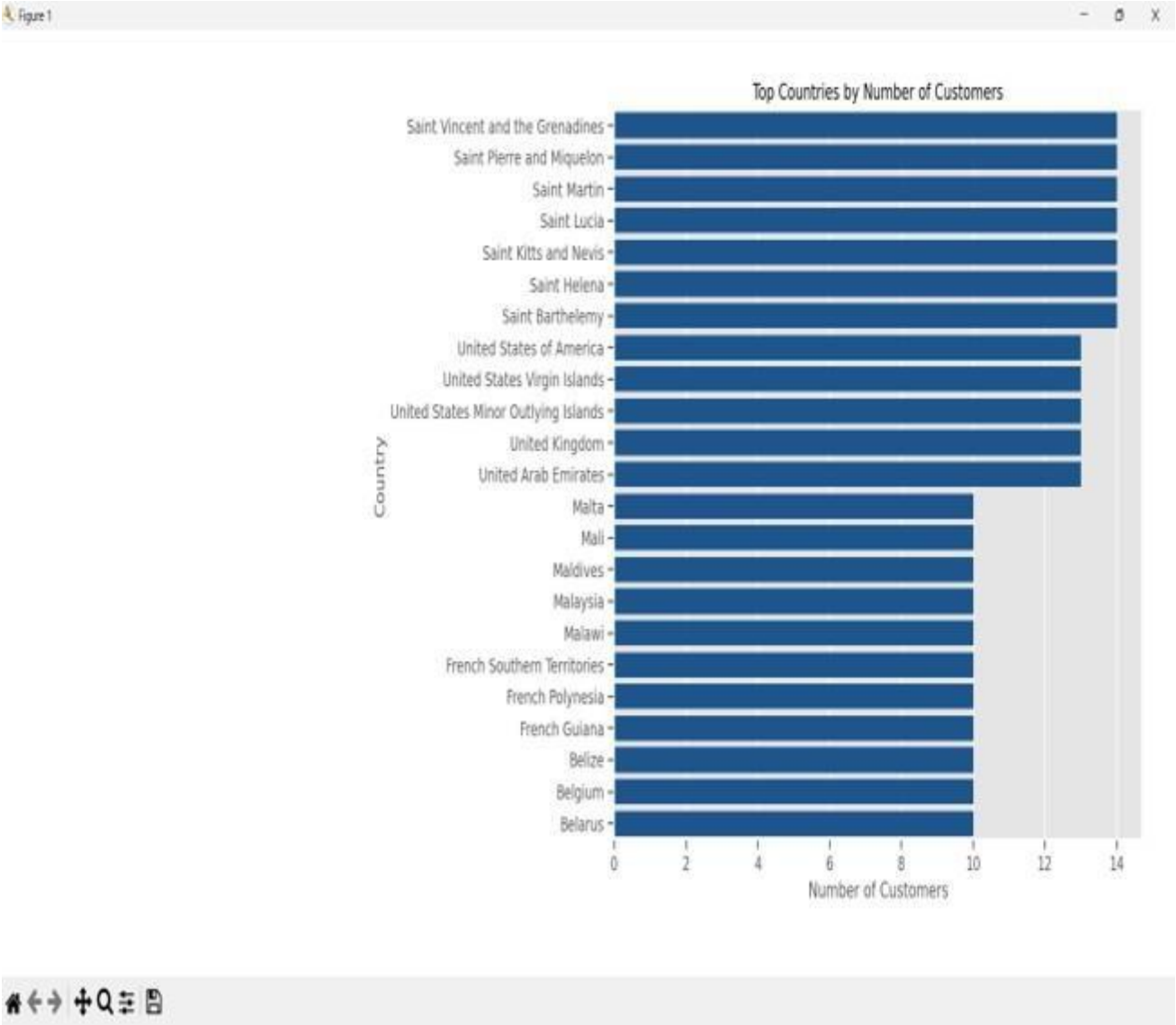


**Figure 2 :** Top Countries by Number of Customers

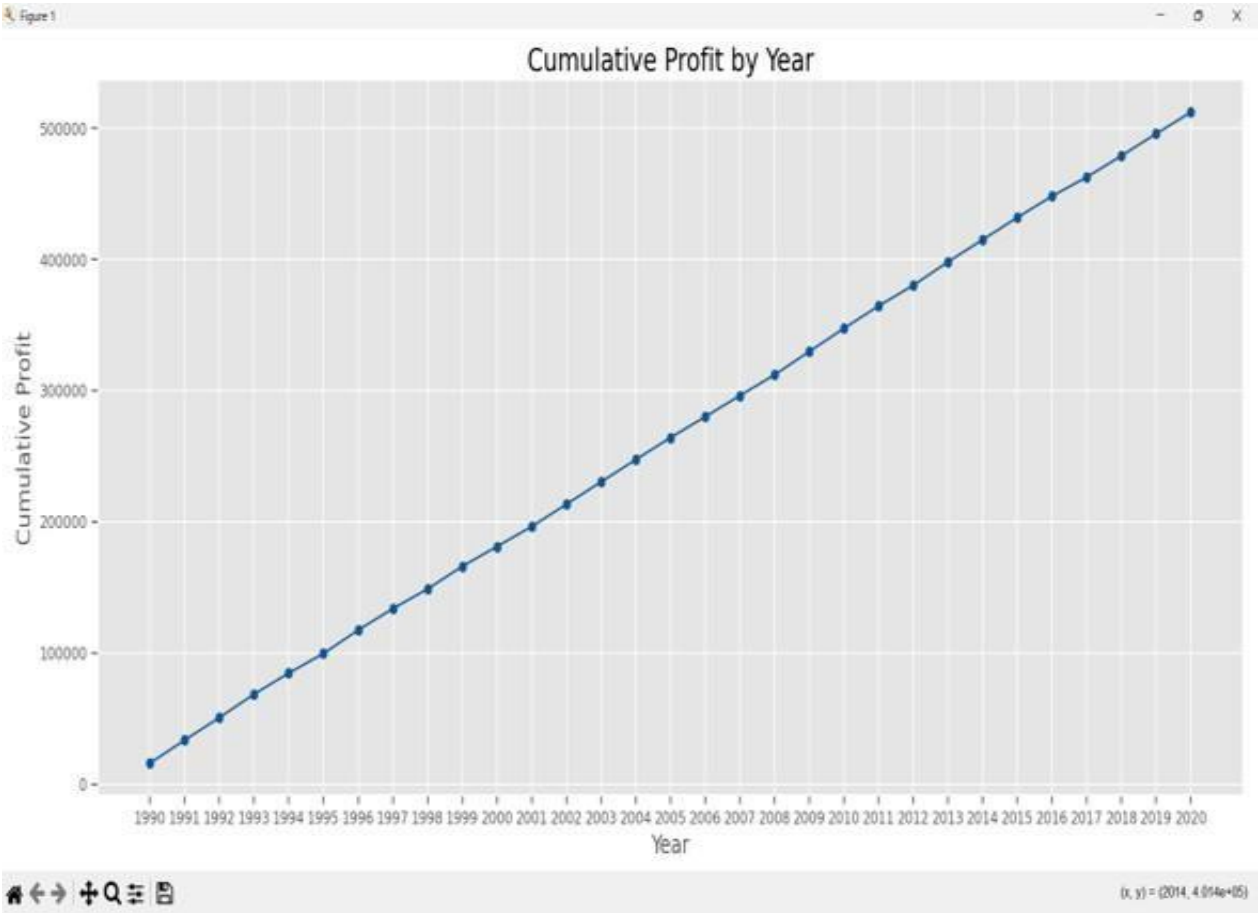


**Figure 3 :** Cumulative Profit by Year

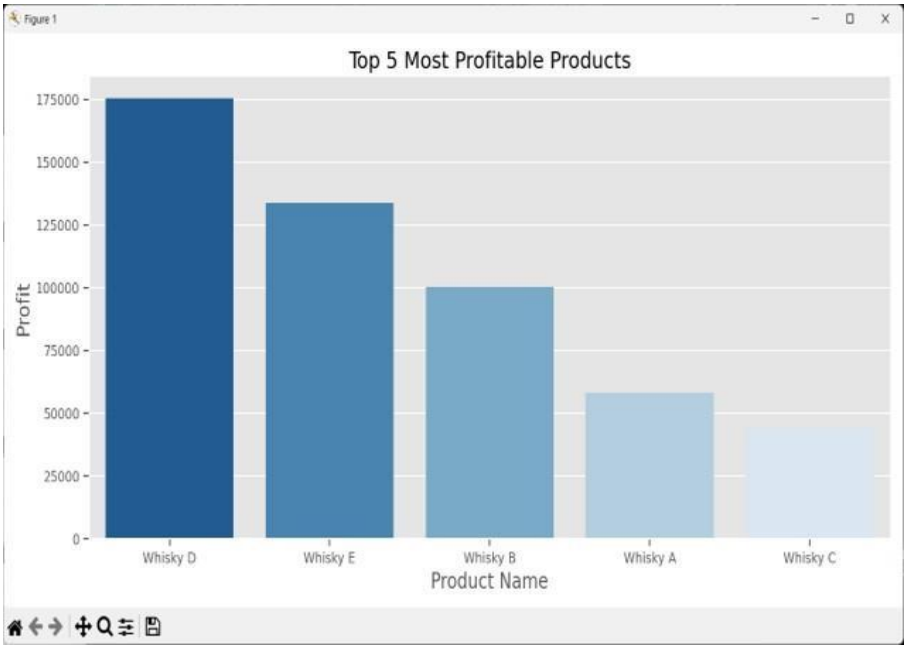


**Figure 4 :** Top 5 Most Profitable Products

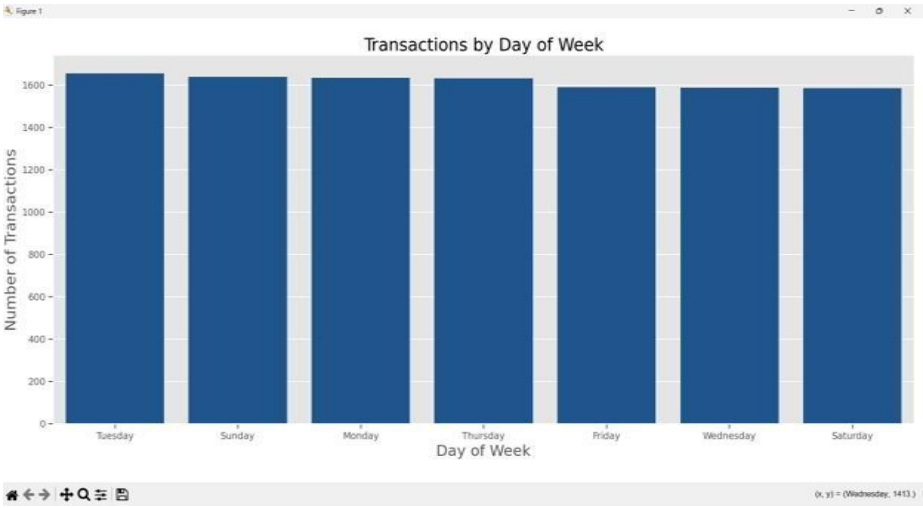


**Figure 5 :** Transactions by Day of Week

# Evaluation Rubrics for Project work:

| Rubric (CO) | Excellent (WT = 3) | Good (WT = 2) | Fair (WT = 1) |
|---|---|---|---|
| ***SELECTION OF TOPIC (CO1)*** | Select a latest topic through complete knowledge of facts and concepts. | Select a topic through partial knowledge of facts and concepts. | Select a topic through improper knowledge of facts and concepts. |
| ***RESEARCH METHODOLOGY (CO1)*** | Framed the research methodology scientifically and validated the same | Framed the research methodology scientifically and validation is not done. | Framed the research methodology unscientifically. |
| ***RESEARCH OBJECTIVE AND PROBLEM STATEMENT (CO2)*** | Thorough definition of research objective and problem statement | Reasonable definition of research objective and problem statement | Improper definition of research objective and problem statement |
| ***ANALYSIS AND SYNTHESIS (CO3)*** | Thorough comprehension through analysis/ synthesis | Reasonable comprehension through analysis/ synthesis | Improper comprehension through analysis/ synthesis |
| ***LITERATURE SURVEY (CO3)*** | Extensive literature survey with standard references | Considerable literature survey with standard references. | Incomplete literature survey with substandard references |
| ***ETHICAL ATTITUDE (CO4)*** | Clearly understands ethical and social practices. | Moderate understanding of ethical and social practices. | Insufficient understanding of ethical and social practices. |
| ***ORAL PRESENTATION (CO5)*** | Presentation in logical sequence with key points, clear conclusion and excellent language | Presentation with key points, conclusion and good language | Presentation with insufficient key points and improper conclusion |
| ***REPORT WRITING (CO5)*** | Project report with clear and logical sequence of chapters using excellent language | Project report with logical sequence of chapters using understandable language | Project report not properly organized |
| ***FINDINGS AND RECOMMENDATIONS(CO6)*** | The findings and recommendations are thoroughly linked to the field of specialization and meet the research objective. | The findings and recommendations are moderately linked to the field of specialization but meet the research objective. | The findings and recommendations is not sufficiently linked to the field of specialization but meets the research objective. |

# ANNEXURE 1

| | |
|---|---|
| **Title of the Project** | Decision Support System Using Data Warehousing for Retail System |
| **Name of the Student** | M Hemesh (22751A0584)<br>M Sai Surya (22751A0590)<br>M Uthaya Kumar (22751A0591)<br>K Sanjay (22751A0563) |
| **Name of the Guide and Designation** | Ms. R. Sathiya Priya, M.Tech, (Ph.D.)<br>Assistant Professor |

**TABLE 1: OUTCOME ATTAINED AND ITS JUSTIFICATION**

| PO | JUSTIFICATION |
|---|---|
| PO1 | Applied foundational concepts of data warehousing and decision support to design and implement a star-schema for retail analytics. |
| PO2 | Analyzed diverse retail data sources (sales, customer, product, employee) and learned how to integrate them through ETL to form a consistent, unified dataset. |
| PO3 | Developed problem-solving skills by building Python scripts for cleansing, transforming, and loading large volumes of transactional data into MySQL. |
| PO4 | Investigated challenges in data quality—such as missing values, duplicate records, and inconsistent formats—and implemented validation rules to ensure high data integrity. |
| PO5 | Practiced professional engineering techniques by modularizing code, documenting scripts, and using version control for collaborative development. |
| PO6 | Gained hands-on experience with modern data-processing tools and frameworks (pandas, SQLAlchemy, PyMySQL) to automate ETL pipelines on a scheduled basis. |
| PO7 | Understood the business impact of analytical insights—such as profit trends and customer segmentation—and how they drive strategic retail decisions. |
| PO8 | Learned best practices for secure data handling and access control by configuring MySQL user roles and encrypted connections for the data warehouse environment. |
| PO9 | Built teamwork and collaboration skills by coordinating code integration, schema design, and dashboard development across multiple contributors. |
| PO10 | Enhanced communication skills by presenting dashboard mock-ups, gathering stakeholder feedback, refining Qlik Sense visualizations based on end-user requirements. |
| PO11 | Developed ability to estimate system resource needs (database sizing, memory for ETL jobs) and to benchmark query performance to meet non-functional requirements. |
| PO12 | Strengthened project management capabilities, planning timelines for ETL development, testing cycles, final deployment, ensuring on-time delivery of the DSS solution. |